



## Laboratorio

Materia: Seminario de lenguajes Python

Carrera: Licenciatura en Sistemas

Docentes: Mg. Lic. María Alejandra Vranic

Lic. Gonzalo Cerbelli

Año cursada: 2026

# Índice

|                                            |          |
|--------------------------------------------|----------|
| <b>1. Objetivo</b>                         | <b>2</b> |
| A. Stack tecnológico obligatorio           | 2        |
| <b>2. Metodología de trabajo en Github</b> | <b>2</b> |
| <b>3. Consignas</b>                        | <b>2</b> |
| A. ABM de Producto                         | 2        |
| B. ABM de Ventas                           | 3        |

# 1. Objetivo

El objetivo de este trabajo práctico es aplicar los conocimientos adquiridos durante la cursada para el desarrollo de una API REST que permita gestionar ventas de productos. El enfoque estará puesto en la utilización del stack propuesto, buenas prácticas de desarrollo de código, manejo de datos, operaciones básicas (CRUD) y generación de reportes.

## A. Stack tecnológico obligatorio

- Python 3.12
- FastAPI
- SQLite
- SQLAlchemy
- Pandas
- Borb

# 2. Metodología de trabajo en Github

Deberán crear un repositorio en GitHub donde alojarán el código de la aplicación, el primer commit deberá ser el proyecto base (configuraciones necesarias para levantar la aplicación) y además deberán actualizar el archivo Readme (lo crean al momento de crear el repositorio) con el nombre del grupo, integrantes y funcionalidades realizadas por cada uno.

Cada nueva funcionalidad deberá ser trabajada en una branch distinta a la main, y mergearse mediante Pull Request, el nombre de las branches con las funcionalidades nuevas tendrá que tener el siguiente formato:

**usuarioGitHub-nombreFuncionalidad.**

Por ejemplo con la funcionalidad del Login: **gonzalocerbelli-endpointCarrito.**

Al momento de *mergear* los cambios de la branch a main pueden utilizar la opción Squash and Merge (compacta todos los commits que tienen la branch en uno solo).

# 3. Consignas

## A. ABM de Producto

Se deberá implementar una API que permita gestionar productos, permitiendo crear un nuevo producto, obtener el listado de productos registrados en la tabla de base de datos, obtener un producto en particular, actualizar los datos de un producto y eliminar un producto. Para ello, los endpoints serán:

- POST /productos
- GET /productos
- GET /productos/{id}
- PUT /productos/{id}
- DELETE /productos/{id}

Los productos deberán contar con los siguientes datos:

- id (int pk autoincrement not null)
- nombre (str not null)
- precio (float not null)

Ejemplo de cuerpo de endpoint POST /producto:

```
{
  "nombre": "Esponja",
  "precio": "11.11"
}
```

Ejemplo de respuesta de endpoint GET /producto/1:

```
{
  "id": 1,
  "nombre": "Esponja",
  "precio": "11.11"
}
```

## B. ABM de Ventas

Se deberán implementar nuevos endpoints para gestionar ventas sobre un sólo producto, permitiendo crear una nueva venta, obtener el listado de ventas registrados en la tabla de base de datos, obtener una venta en particular, actualizar los datos de una venta y eliminar una venta. Para ello, los endpoints serán:

- POST /ventas
- GET /ventas
- GET /ventas/{id}
- PUT /ventas/{id}
- DELETE /ventas/{id}

Las ventas deberán contar con los siguientes datos:

- id (int pk autoincrement not null)
- fecha (date not null)
- hora (time not null)
- id\_producto (fk int not null)
- cantidad (int not null)
- precio\_total (float not null)

El precio total de la venta será calculado por la API tomando el precio unitario del producto y la cantidad de la venta.

Ejemplo de cuerpo de endpoint POST /ventas:

```
{
  "fecha": "2000-01-01",
  "hora": "10:30",
  "id_producto": 2,
  "cantidad": 15
}
```

Ejemplo de cuerpo de endpoint GET /ventas/1:

```
{
  "id": 1,
  "fecha": "2000-01-01",
  "hora": "10:30",
  "producto": {
    "id": 2,
    "nombre": "Arroz",
    "precio": 300
  },
  "cantidad": 15,
  "precio_total": 4500
}
```